

BGE-M3：稀疏嵌入、密集嵌入和多向量嵌入

1. BGE-M3 如何生成稀疏嵌入向量？

BGE-M3 (BAAI General Embedding – Multi-Functionality, Multi-Linguality, Multi-Granularity) 是一个多功能嵌入模型，能够同时生成稀疏嵌入、密集嵌入和多向量嵌入 (ColBERT 风格)。下面是它生成稀疏嵌入的原理和过程：

原理：

- **稀疏嵌入的核心：**稀疏嵌入捕捉文本中的关键词特征，每个非零维度对应一个具体的 token (通常是词或子词)，其值表示该 token 的重要性 (类似 BM25 的权重)。

BGE-M3 的方法：BGE-M3 通过神经网络 (基于 Transformer) 对输入文本进行编码，输出一个稀疏向量，其中：

- 维度与模型的词表 (vocabulary) 大小一致。
- 每个维度对应词表中的一个 token，非零值表示该 token 在文本中的重要性 (类似于加权分数)。
- 重要性分数是通过模型学习得到的，而不是像 BM25 那样基于简单的词频 (TF) 和逆文档频率 (IDF) 公式。BGE-M3 结合了神经网络的语义理解能力，生成的权重更能捕捉上下文相关性。

生成过程：

输入文本：

- a. 例如：“**猢狲施展烈焰拳，击退妖怪；随后开启金刚体，抵挡神兵攻击。**”
- b. 文本会被分词 (tokenize)，映射到 BGE-M3 的词表中 (通常基于 SentencePiece 或类似的分词器)。

1.

编码：

- a. BGE-M3 使用 Transformer 架构对输入 token 序列进行编码，生成每个 token 的上下文表示。
- b. 对于稀疏嵌入，模型会输出一个高维向量 (维度等于词表大小，通常是几十万维)，但只有与输入文本相关的 token 对应的维度有非零值。
- c. 非零值是通过模型的最后一层 (通常是一个线性层加激活函数，如 ReLU) 计算得到的，表示该 token 的语义权重。

1.

稀疏向量输出：

输出是一个字典形式，键是词表中 token 的索引 (或 ID)，值是对应的权重。例如：

python

Copy

```
sparse_vecs = { 103 : 0.85 , 302 : 0.62 , 567 : 0.33 , ... }
```

- a. 这里 103 可能对应“猢猻”，302 对应“烈焰拳”，权重反映它们在上下文中的重要性。
- b. 未出现在文本中的 token 对应的维度值为 0，因此向量非常稀疏。

与 BM25 的区别：

- BM25：权重基于明确的统计公式（TF 和 IDF），维度直接对应词表，计算简单但缺乏深层语义。
 - BGE-M3 稀疏嵌入：通过神经网络学习权重，捕捉更复杂的语义关系（例如，“烈焰拳”和“攻击”可能在语义上有隐含关联），但计算复杂度更高。
-

2. BGE-M3 混合嵌入的维度是多少？

BGE-M3 的混合嵌入指的是将稀疏嵌入、密集嵌入和多向量嵌入（ColBERT 风格）结合使用，以提升检索的准确性和灵活性。混合嵌入本身并不是一个单一的向量，而是多种嵌入的组合，因此“维度”需要分别讨论：

（1）稀疏嵌入的维度：

维度：等于 BGE-M3 模型的词表大小（vocabulary size）。

- 对于 BGE-M3，词表大小通常是几万到几十万维（具体取决于使用的 tokenizer，例如 SentencePiece）。
 - 示例：如果词表有 305,000 个 token，那么稀疏向量的理论维度是 305,000，但实际存储的只有非零维度（通常几十到几百个）。
- **特点：**只有少量维度有值（对应文本中的 token），存储高效，适合关键词匹配。

（2）密集嵌入的维度：

维度：固定且较小，通常是模型配置中定义的隐藏层大小。

- 对于 BGE-M3，密集嵌入的默认维度是 1024（也可能根据模型变体调整，例如 768 或 512）。
 - 示例：dense_vecs = [0.8, -0.2, 0.5, ..., 0.3]（共 1024 个值）。
- **特点：**每个维度都有值，捕捉全局语义，适合语义相似性计算。

（3）多向量嵌入（ColBERT 风格）的维度：

维度：每个 token 对应一个固定维度的向量（通常是 128 或 768），整个文本生成一组向量。

- 示例：如果文本有 50 个 token，每个 token 的向量是 128 维，那么 ColBERT 嵌入是一个 50×128 的矩阵。
 - BGE-M3 的 ColBERT 嵌入通常用于精细化的 token 级匹配。
- **特点：**适合需要 token 级交互的场景（如晚期交互检索）。

（4）混合嵌入的维度：

混合嵌入的含义：BGE-M3 并不直接输出一个“混合向量”，而是提供稀疏、密集和 ColBERT 三种嵌入的组合，开发者可以根据任务选择如何融合。

- **维度不统一：**稀疏嵌入是高维稀疏向量，密集嵌入是低维稠密向量，ColBERT 是多向量矩阵。它们通常不会合并成单一向量，而是分别用于检索的不同阶段。

融合方式：

- **检索阶段：**稀疏嵌入用于快速关键词匹配，密集嵌入用于语义相似性排序，ColBERT 用于 token 级精细重排序。
- **实际维度：**取决于融合后的评分方式。例如，最终可能输出一个标量分数（基于加权或点积），而不是固定维度的向量。

示例：在你的战斗日志场景中：

- 稀疏嵌入：{103: 0.85, 302: 0.62, ...}（维度约 305,000，实际非零值几十个）。
- 密集嵌入：[0.8, -0.2, ..., 0.3]（1024 维）。
- ColBERT 嵌入：一个矩阵（如 50×128 ，如果日志有 50 个 token）。
- 混合检索：可能先用稀疏嵌入过滤出包含“烈焰拳”的日志，再用密集嵌入排序语义相关性，最后用 ColBERT 确保 token 级匹配（如“烈焰拳”和“妖怪”是否紧密相关）。

3. 在“灭神纪·獠狮”场景中的应用

以战斗日志为例，假设你想检索所有“使用烈焰拳击败妖怪”的日志：

稀疏嵌入：

- BGE-M3 会为日志生成稀疏向量，突出“烈焰拳”“妖怪”等关键词的权重。
- 维度：约 305,000（词表大小），但只有几十个非零值（如 {烈焰拳: 0.85, 妖怪: 0.62, ...}）。
- 作用：快速过滤包含“烈焰拳”和“妖怪”的日志。

密集嵌入：

- 生成 1024 维向量，捕捉日志的整体语义（例如“战斗激烈”或“胜利”）。
- 作用：对过滤后的日志按语义相似性排序，确保与查询“击败妖怪”的意图匹配。

混合嵌入：

- 结合稀疏和密集嵌入，先用稀疏向量高效召回相关日志，再用密集向量排序。

- 如果启用 CoBERT，还可以检查“烈焰拳”和“妖怪”是否在日志中紧密相关（例如出现在同一句话）。
 - 维度：稀疏（~305,000，实际非零几十个）+ 密集（1024）+ CoBERT（矩阵形式，视 token 数而定）。
-

输出示例：

稀疏嵌入（前 10 个非零值）：[(103, 0.85), (302, 0.62), (567, 0.33), ...]

密集嵌入（前 10 维）：[0.8, -0.2, 0.5, 0.3, -0.1, 0.4, 0.2, -0.3, 0.6, 0.1]

密集嵌入维度：1024

4. 实际应用优势

- **混合检索（Hybrid Retrieval）**

结合稠密与稀疏检索（或三者联合），在公开评测（如MIRACL、MKQA）中显著超越单一模式。例如，BGE-M3（ALL）在三项评测中全面领先OpenAI等模型。

- **动态适配场景**

- 多语言场景：稠密检索主导跨语言语义匹配；
- 长文档场景：稀疏检索捕捉关键词重要性；
- 高精度需求：多向量检索提升细粒度排序。

5. 总结

- BGE-M3 如何生成稀疏嵌入？通过 Transformer 编码输入文本，生成高维稀疏向量（维度等于词表大小，约几十万维）。
- 只有与文本相关的 token 对应维度有非零值，权重由模型学习，反映 token 的语义重要性。
- **混合嵌入的维度？**
 - 稀疏嵌入：理论维度几十万，实际非零值几十到几百。
 - 密集嵌入：1024 维（固定）。
 - CoBERT 嵌入：矩阵形式（token 数 × 128 或 768）。
 - 混合嵌入是多种向量的组合，维度不统一，通常用于分阶段检索而非单一向量。